Kahn networks, 50 years later

Marc Pouzet

Team Parkas
ENS/PSL/INRIA, Paris

Marc.Pouzet@ens.fr

INRIA Paris
Dec. 16, 2024

In 1974, Gilles Kahn published a short, five pages, paper [Kahn, 1974].

INFORMATION PROCESSING 74 – NORTH-HOLLAND PUBLISHING COMPANY (1974)

THE SEMANTICS OF A SIMPLE LANGUAGE FOR PARALLEL PROGRAMMING

Gilles KAHN

IRIA-Laboria, Domaine de Voluceau, 78150
Rocquencourt, France

and

Commissariat à l'Energie Atomique, France

In this paper, we describe a simple language for parallel programming. Its semantics is studied thoroughly. The desirable properties of this language and its deficiencies are exhibited by this theoretical study. Basic results on parallel program schemata are given. We hope in this way to make a case for a more formal (i.e. mathematical) approach to the design of languages for systems programming and the design of operating systems.

There is a wide disagreement among systems designers as to what are the best primitives for writing systems programs. In this paper, we describe a simple language for parallel programming and study its mathematical properties.

1. A SIMPLE LANGUAGE FOR PARALLEL PROGRAMMING.

The features of our mini-language are exhibited on the sample program S on fig.1. The conventions are close to Algol and we only insist upon the new features. The program S consists of a set of declarations and a body. Variables of type *integer channel* are declared at line (1), and for any simple type σ (boolean, real, etc...) we could have declared a σ *channel*. Then processes f, g and h are declared, much like procedures. Aside from usual parameters (passed by value in this example, like INIT at line (3)), we can declare in the heading of the process how it is linked to other processes : at line (2) f is stated to communicate via two input lines that can carry integers, and one similar output line.
The body of a process is an usual Algol program except for invocation of *wait* on an input line (e.g. at (4)) or *send* a variable *on* a line of compatible type (e.g. at (5)). The process stays blocked on a *wait* until something is being sent on this line by another process, but nothing can prevent a process from performing a *send* on a line.
In other words, processes communicate via first-in first-out (fifo) queues.
Calling instances of the processes is done in the body of the main program at line (6) where the actual names of the channels are bound to the formal parameters of the processes. The infix operator *par*

```
 Begin
(1) Integer channel X, Y, Z, T1, T2 ;
(2) Process f(integer  in U,V; integer out W) ;
    Begin integer I ; logical B ;
          B := true ;
          Repeat Begin
(4)         I := if B then wait(U) else wait(V) ;
(7)         print (I) ;
(5)         send I on W ;
            B := ¬B ;
            end ;
    End ;
  Process g(integer in U ; integer out V, W) ;
    Begin integer I ; logical B ;
      B := true ;
      Repeat Begin
        I := wait (U) ;
        if B then send I on V else send I on  W ;
        B := ¬B ;
        End ;
    End ;
(3) Process h(integer in U;integer out V; integer INIT);
    Begin integer I ;
      send INIT on V ;
      Repeat Begin
        I := wait(U) ;
        send I on V ;
        End ;
    End ;
  Comment : body of mainprogram ;
(6) f(Y,Z,X) par g(X,T1,T2) par h(T1,Y,0) par h(T2,Z,1
  End ;
```

Fig.1. Sample parallel program S.

Gilles Kahn was interested in the semantics and implementation of systems running in parallel and interacting with each others.

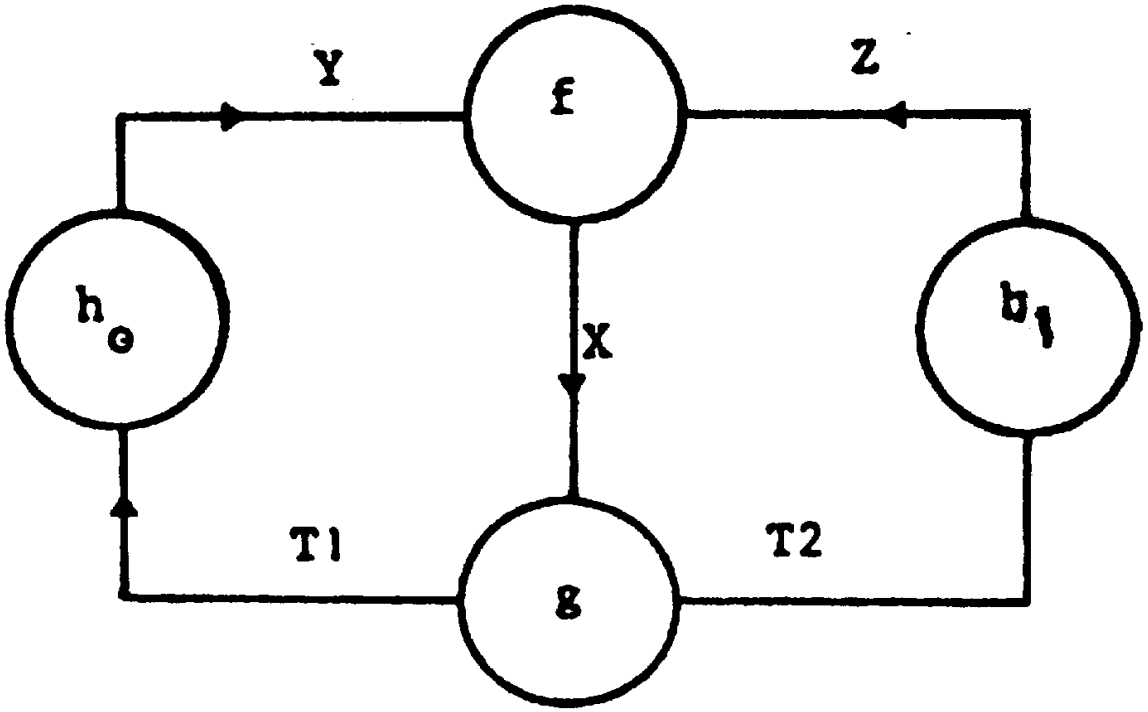
```
 Begin
(1) Integer channel X, Y, Z, T1, T2 ;
(2) Process f(integer  in U,V; integer out W) ;
    Begin integer I ; logical B ;
          B := true ;
          Repeat Begin
(4)         I := if B then wait(U) else wait(V) ;
(7)         print (I) ;
(5)         send I on W ;
            B := ¬B ;
            end ;
    End ;
   Process g(integer in U ; integer out V, W) ;
    Begin integer I ; logical B ;
      B := true ;
      Repeat Begin
        I := wait (U) ;
        if B then send I on V else send I on  W ;
        B := ¬B ;
        End ;
    End ;
(3) Process h(integer in U;integer out V; integer INIT);
    Begin integer I ;
      send INIT on V ;
      Repeat Begin
        I := wait(U) ;
        send I on V ;
        End ;
    End ;

  Comment : body of mainprogram ;

(6) f(Y,Z,X) par g(X,T1,T2) par h(T1,Y,0) par h(T2,Z,1

  End ;
```

It can be represented by a graph, where nodes represent processes and arcs are the communication channels between these processes.



Basic concepts

- A set of deterministic (sequential) programs that run in parallel.
- Each program has his own memory.
- Two communication primitives:
  - `wait(U)` reads one data from input channel `U`;
  - `send I to W` writes the value in `I` on the output channel `W`.
- Communication channels are the only way processes can communicate with each other.
- A communication channel transmits information within an impredictable but finite amount of time.

FIFO communication

- "A process is blocked on a wait until something is being sent";
- "nothing can prevent a process from performing a send";
- that is, "processes communicate via first-in first-out (FIFO) queues".
- "Edges are thought as pipes; able to carry data of a given type D (i.e., integer, boolean, pointer, procedure etc...)"
- Execution is asynchronous.
- The size of FIFOs is unbounded.
- A single writter but several readers (duplication).
- Operations like "select" (that is, `wait(U or U')`) are forbidden.

Semantics

A sequential process, with possible internal state, is interpreted as a function from the history of its inputs to the history of its outputs.

What is an history?
How to model a process?

Semantics

A sequential process, with possible internal state, is interpreted as a function from the history of its inputs to the history of its outputs.

What is an history?

How to model a process?

Semantics

A sequential process, with possible internal state, is interpreted as a function from the history of its inputs to the history of its outputs.

What is an history?
How to model a process?

Streams and Continuous Mappings

Streams

- $D^\omega = D^* \cup V^\infty$ the set of finite and infinite sequences of elements in D
- Λ is the empty sequence.
- $\subseteq$ is the prefix order between sequences: $X \subseteq Y$ iff X is a prefix of Y.
- Λ is minimal for the prefix order.
- D^ω is a Complete Partial Order: any increasing chain $a = X_1 \subseteq ... \subseteq X_n \subseteq ...$ has a least upper bound $\lim_{D^\omega}(a)$

Continuous Mappings

- f is monotone iff $\forall x, y.(x \subseteq y) \Rightarrow (f(x) \subseteq f(y))$
- A mapping f from CPO A to CPO B is continuous iff for any increasing chain a of A:

$$f(\lim_A(a)) = \lim_B(f(a))$$

Examples

The paper gives some examples of continuous functions.

- F (for first). It returns the leftmost element of its input stream:

$$F(x.xs) = x$$

- R (for remainder). It returns the right of its leftmost element:

$$R(x.xs) = xs$$

- A (for append). It returns the leftmost element of its input followed by its second input.

$$A(x.xs)\,ys = x.ys$$

Generic primitives [Caspi, 1992, Caspi and Pouzet, 1996]

$$
\begin{array}{lcl}
const(v) & = & v.const(v) \\
extend(f.fs)(v.s) & = & (f\ v).extend(fs)(s) \\
when(v.s, 1.c) & = & v.when(s, c) \\
when(v.s, 0.c) & = & when(s, c) \\
merge(1.c, v.s_1, s_2) & = & v.merge(c, s_1, s_2) \\
merge(0.c, s_1, v.s_2) & = & v.merge(c, s_1, s_2) \\
(v_1.s_1)\ fby\ s_2 & = & v_1.s_2 \\
next\ (x.s) & = & s \\
restart\ f\ x & = & (f\ x)\ fby\ (restart\ f\ (next\ (x))) \\
 & = & \Lambda \text{ otherwise}
\end{array}
$$

restart f . resets the application of f at every instant. It turns a sequential function into a combinatorial one.

The Kahn Principle

A computing node with n inputs carrying data in $D_1, D_2, ...$ and p outputs carrying data in $D'_1, D'_2, ...$ is interpreted as p continuous mappings $f_1, ..., f_p$:

$$f_i : D_1^\omega \times ... \times D_n^\omega \mapsto D_i'^\omega$$

- Monotonicity: the output can be produced before all inputs are available. Giving more input does not change the already produced outputs.
- Continuity: a computing node cannot decide to produce an output once it has received an infinite number of inputs.

Systems of equations

A parallel program is interpreted as a system of equations.
For a node f, interpreted by the functions $f_1, ..., f_p$, with input variables $X_1, ..., X_n$ and output variables $X'_1, ..., X'_p$, define the equation:

$$X'_i = f_i(X_1, ..., X_n)$$

The semantics of a Kahn network is the least fixpoint of the associated system of equations.

Given a system of n equations:

$$X_i = g_i(\vec{X})$$

where g_i are continuous. Its minimal solution is $lim_{j \to \infty}(\vec{X}^j)$ where:

$$\begin{array}{lcl} X_i^0 & = & \Lambda \\ X_i^{j+1} & = & g_i(\vec{X}^j) \end{array}$$

Recursion

This model allows for describing recursively defined networks.

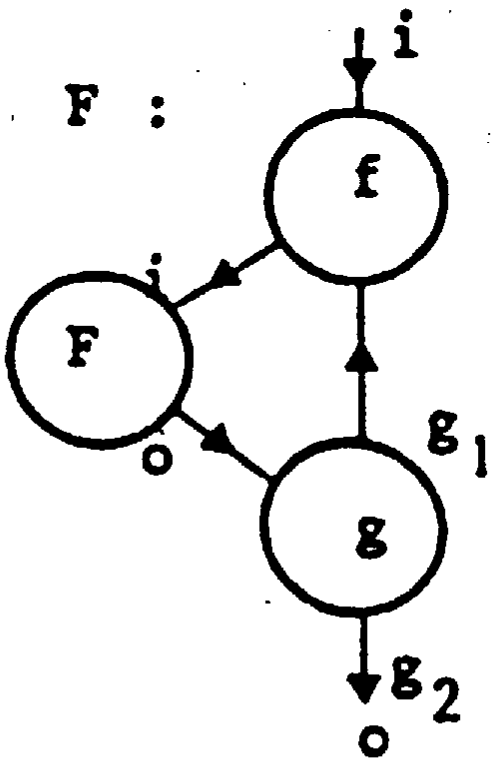


Fig .8. A recursive parallel schema.

Example: sieve of Eratosthenes [Kahn and MacQueen, 1977]

```
Process INTEGERS out QO;
    Vars N; 1 → N;
    repeat INCREMENT N; PUT(N,QO) forever
Endprocess;

Process FILTER PRIME in QI out QO;
    Vars N;
    repeat GET(QI) → N;
           if (N MOD PRIME)≠ ∅ then PUT(N,QO) close
    forever
Endprocess;

Process SIFT in QI out QO;
    Vars PRIME; GET(QI) → PRIME;
    PUT (PRIME,QO); comment emit a discovered prime;
    doco channels Q;
     FILTER(PRIME,QI,Q); SIFT(Q,QO)
    closeco
Endprocess;

Process OUTPUT in QI; Comment this is a library process;
    repeat PRINT(GET(QI)) forever
Endprocess;

Start doco channels Q1 Q2;
    INTEGERS(Q1); SIFT(Q1,Q2); OUTPUT(Q2);
    closeco;
```

Fig.3. Sieve of Eratosthenes.

Data-flow, coroutine and functional programming [1]

In 1977, Kahn and MacQueen propose several ways for implementing this language, in particular using coroutines [Kahn and MacQueen, 1977].

At the same period, lazy functional languages started to emerge (e.g., Miranda).

It is now possible to program by directly writting stream programs and streams equations, e.g., in Haskell.

and rely on lazy evaluation.

[1] "Kahn networks at the dawn of functional programming", by David MacQueen, 2009 [MacQueen, 2009]; A "Data flow Retrospective", Jack Dennis (`https://csg.csail.mit.edu/Dataflow/talks/DennisTalk.pdf`)

Data-flow, coroutine and functional programming [1]

In 1977, Kahn and MacQueen propose several ways for implementing this language, in particular using coroutines [Kahn and MacQueen, 1977].

At the same period, lazy functional languages started to emerge (e.g., Miranda).

It is now possible to program by directly writting stream programs and streams equations, e.g., in Haskell.

and rely on lazy evaluation.

[1] "Kahn networks at the dawn of functional programming", by David MacQueen, 2009 [MacQueen, 2009]; A "Data flow Retrospective", Jack Dennis (`https://csg.csail.mit.edu/Dataflow/talks/DennisTalk.pdf`)

Data-flow, coroutine and functional programming [1]

In 1977, Kahn and MacQueen propose several ways for implementing this language, in particular using coroutines [Kahn and MacQueen, 1977].

At the same period, lazy functional languages started to emerge (e.g., Miranda).

It is now possible to program by directly writting stream programs and streams equations, e.g., in Haskell.

and rely on lazy evaluation.

[1] "Kahn networks at the dawn of functional programming", by David MacQueen, 2009 [MacQueen, 2009]; A "Data flow Retrospective", Jack Dennis (`https://csg.csail.mit.edu/Dataflow/talks/DennisTalk.pdf`)

Data-flow, coroutine and functional programming [1]

In 1977, Kahn and MacQueen propose several ways for implementing this language, in particular using coroutines [Kahn and MacQueen, 1977].

At the same period, lazy functional languages started to emerge (e.g., Miranda).

It is now possible to program by directly writting stream programs and streams equations, e.g., in Haskell.

and rely on lazy evaluation.

[1] "Kahn networks at the dawn of functional programming", by David MacQueen, 2009 [MacQueen, 2009]; A "Data flow Retrospective", Jack Dennis (`https://csg.csail.mit.edu/Dataflow/talks/DennisTalk.pdf`)

Kahn networks

(+): **Conciliate parallelism and determinism:** a process defines a function; several forms of executions are possible, sequential or parallel; they all compute the same streams.

(+): **Modularity:** continuity is closed by composition and fixpoint; a network is a continuous function

(+): **Asynchronous distributed execution:** no central scheduler is necessary.

(+/-): **Time independence:** no explicit timing; but impossible to state that two events happen at the same time.

(-): **Ressources:** Kahn networks can run with unbounded memory and have deadlock. The composition of two bounded memory/deadlock-free networks may not be bounded memory/deadlock-free.

Kahn networks at the dawn of functional and synchronous programming

Examples (with Haskell, Coq and ScadeOne)

Conclusion

An extremely influencial work.

It uniquely conciliate parallelism and determinacy.

Synchronous functional languages, e.g., Lustre and relatives, are all based on a Kahnian semantics.

Kahn networks must be restricted to those that can be executed synchronously; this is statically checked with dedicated typing.

They can be translated into statically scheduled sequential code that run in bounded time and space; with no deadlock.

The Kahn model have been (and is) very popular for architectural design, following the pionnering work of Jack Dennis and his group at MIT.

An more widely, stream programming.

Un dernier mot pour la route...

In the conclusion of his paper, Gilles Kahn wrote:

> Our last conclusion is to recall a principle that has been so often fruitful in Computer Science and that is central to Scott's theory of computation : a <u>good</u> concept is one that is closed
>
> 1. under arbitrary composition
> 2. under recursion.

References I

Caspi, P. (1992).
Clocks in dataflow languages.
Theoretical Computer Science, 94:125–140.

Caspi, P. and Pouzet, M. (1996).
Synchronous Kahn Networks.
In *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, Philadelphia, Pensylvania.

Kahn, G. (1974).
The semantics of a simple language for parallel programming.
In *IFIP 74 Congress*. North Holland, Amsterdam.

Kahn, G. and MacQueen, D. B. (1977).
Coroutines and networks of parallel processes.
In *IFIP Congress*, pages 993–998.

MacQueen, D. B. (2009).
From Semantics to Computer Science Essays in Honour of Gilles Kahn, chapter Kahn networks at the dawn of functional programming.
Cambridge University Press.